

0919 Answer | 同一位模式经过类型、寄存器和指令

原题 7 道 · 0918 的数据边界 · 能力账本

本组用“存的是哪几位、按什么类型解释、指令读写哪一处”串题。题中的机器位宽和内存模型优先于真实 C 编译器的其他平台行为。讲解已核原题，限时独立未验。

考场入口

题	第一笔	关键结果
01	134=86H, 246=F6H, 先存8位再解释	R1=86H、R5=90H、R6=7CH
02	n=0 时 unsigned n-1 下溢；float 有24位有效精度	f1最大正确 n30、f2无溢出 n126
03	call 的位移相对下一指令 0040102AH	-42=FFFFFFD6H, 小端字节 D6 FF FF FF
04	16位字、20位地址、8位数据总线分开	ALU16、MAR20、MDR8
05	先算机器码长度与下一条地址	第20条 004010B9H
06	a[5] 从基址加20字节	0013E004H、值 FFFFECDCH、sum E H
07	被除数先符号扩展入 R:Q；零除数与商越界分别判	源图中 idiv 操作数不一致，按 R2 的 x 解

01 | 2011-43 : 8 位截断之后再解读 signed

43. (11 分) 假定在一个 8 位字长的计算机中运行如下 C 程序段:

```
unsigned int x=134;
unsigned int y=246;
int m=x;
int n=y;
unsigned int z1=x-y;
unsigned int z2=x+y;
int k1=m-n;
int k2=m+n;
```

若编译器编译时将 8 个 8 位寄存器 R1~R8 分别分配给变量 x、y、m、n、z1、z2、k1 和 k2。

请回答下列问题。(提示: 带符号整数用补码表示。)

- (1) 执行上述程序段后, 寄存器 R1、R5 和 R6 的内容分别是什么(用十六进制表示)?
- (2) 执行上述程序段后, 变量 m 和 k1 的值分别是多少(用十进制表示)?
- (3) 上述程序段涉及带符号整数加/减、无符号整数加/减运算, 这四种运算能否利用同一个加法器辅助电路实现? 简述理由。
- (4) 计算机内部如何判断带符号整数加/减运算的结果是否发生溢出? 上述程序段中, 哪些带符号整数运算语句的执行结果会发生溢出?

(1) 寄存器。 $x=134=86H$, $y=246=F6H$; $m=x$ 和 $n=y$ 赋给 8 位 signed int 后, 在题设补码机器模型中位型未变, 但解释为 $m=-122$ 、 $n=-10$ 。无符号 $z1=x-y=-112 \bmod 256=90H$, $z2=x+y=380 \bmod 256=7CH$ 。所以 $R1=86H$ 、 $R5=90H$ 、 $R6=7CH$ 。

(2) m 与 k1。 $m=86H$ (按 signed 值 -122) ; $k1=m-n=-122-(-10)=-112=90H$ 。题要“用十六进制”, 两个位型分别 $86H$ 、 $90H$ 。

(3) 一套加法器。减法经 $A+(\sim B)+1$, 有符号和无符号共用 8 位 ALU, 差别是结果/标志怎样解释: 无符号借位/进位看 CF; signed 范围看 OF。

(4) OF。加法同号相加却得到异号; 减法异号相减却得到与被减数异号。 $k1=-112$ 在 $-128..127$ 内, 不溢出; $k2=m+n=-132$ 超出, 8 位回绕为 $7CH$, 溢出。 $z1, z2$ 是 unsigned 模 256 运算, 不用 signed OF 裁判它们。

先保存位, 再读数值

$F6H$ 当 unsigned 是 246、当 signed 是 -10 ; $86H$ 当 unsigned 是 134、当 signed 是 -122 。不能把 $m=x$ 直接说成数学整数 134 再做 signed 运算。本题按题设 8 位补码机器的截断模型说明; 标准 C 对不可表示的 unsigned $\rightarrow$ signed 转换允许实现定义处理。

02 | 2017-43：整数上界与 float 精度是两道关

43. (13 分) 已知 $f(n) = \sum_{i=0}^n 2^i = 2^{n+1} - 1 = \overbrace{11 \cdots 1}^{n+1 \text{ 位}} \text{B}$, 计算 $f(n)$ 的 C 语言函数 f1 如下:

```
int f1(unsigned n){
    int sum=1, power=1;
    for(unsigned i=0;i<=n-1;i++){
        power *= 2;
        sum += power;
    }
    return sum;
}
```

将 f1 中的 int 都改为 float, 可得到计算 $f(n)$ 的另一个函数 f2。假设 unsigned 和 int 型数据都占 32 位, float 采用 IEEE 754 单精度标准。请回答下列问题。

(1) 当 $n=0$ 时, f1 会出现死循环, 为什么? 若将 f1 中的变量 i 和 n 都定义为 int 型, 则 f1 是否还会出现死循环? 为什么?

(2) f1(23)和 f2(23)的返回值是否相等? 机器数各是什么(用十六进制表示)?

(3) f1(24)和 f2(24)的返回值分别为 33 554 431 和 33 554 432.0, 为什么不相等?

(4) $f(31) = 2^{32} - 1$, 而 f1(31)的返回值却为-1, 为什么? 若使 f1(n)的返回值与 $f(n)$ 相等, 则最大的 n 是多少?

(5) f2(127)的机器数为 7F80 0000H, 对应的值是什么? 若使 f2(n)的结果不溢出, 则最大的 n 是多少? 若使 f2(n)的结果精确(无舍入), 则最大的 n 是多少?

(1) $n=0$ 的循环。n 为 unsigned, $n-1$ 得 FFFFFFFFH; i 从 0 递增后回绕, $i \leq \text{FFFFFFFH}$ 永真, 故死循环。若 i, n 都为 signed int, $0-1=-1$, 第一次 $i=0 \leq -1$ 为假, 循环不执行并返回 1, 即 $f(0)$ 。

(2) $n=23$ 。数学值 $2^{24}-1=00\text{FFFFFFH}$, 32 位 int 可精确存; float 单精度可精确表示 24 位有效二进制数, 因此 f1 与 f2 返回的 int 都为 00FFFFFFH。

(3) $n=24$ 。f1 的整数 $2^{25}-1=33554431$; float 在此量级相邻可表示数间距为 2, 逐次运算后舍入到 33554432.0。f2 返回值发生差异是精度舍入, 不是浮点溢出; 返回 int 时得到 33554432。

(4) $n=31$ 与 int 最大可行 n。 $2^{32}-1$ 的低 32 位为 FFFFFFFFH, 按题设机器回绕解释为 -1。signed int 可表示的最大该形数是 $2^{31}-1$, 即 $n=30$; 再大即越界。严格 C 语义下有符号溢出未定义, 不能把机器回绕当成可移植 C 保证。

(5) f2(127)。7F800000H 是 IEEE754 正无穷大; 到 $n=127$ 时理论值临近 2^{128} , 舍入/溢出到正无穷。保持有限不溢出的最大 $n=126$; 完全精确无舍入的最大 $n=23$ 。还要防 $n=0$ 的 unsigned 循环问题, 这三个上界都是在 $n>0$ 正常执行的前提下谈数值。

三个“坏掉”分别在哪一步

循环边界 $n-1$ 的类型决定能否结束; 整数结果超过有符号范围决定 f1 是否还能表示; float 即使没溢出, 也可能因 24 位有效精度先丢掉尾部的 1。不要把精度损失叫成无穷大, 也不要因 f2(127) 推出 f2(24) 已溢出。

03 | 2019-45：递归调用、相对位移与乘法溢出

45. (16 分) 已知 $f(n) = n! = n \times (n-1) \times (n-2) \times \dots \times 2 \times 1$ ，计算 $f(n)$ 的 C 语言函数 `f1` 的源程序（阴影部分）及其在 32 位计算机 M 上的部分机器级代码如下：

int f1(int n){		
1 00401000	55	push ebp
...	...	...
if(n>1)		
1100401018	83 7D 08 01	cmp dword ptr [ebp+8],1
120040101C	7E 17	jle f1+35h (00401035)
return n*f1(n-1);		
130040101E	8B 45 08	mov eax, dword ptr [ebp+8]
1400401021	83 E8 01	sub eax, 1
1500401024	50	push eax
1600401025	E8 D6 FF FF FF	call f1 (00401000)
...	...	...
1900401030	0F AF C1	imul eax, ecx
2000401033	EB 05	jmp f1+3Ah (0040103a)
else return 1;		
2100401035	B8 01 00 00 00	mov eax,1
}		
...	...	...
2600401040	3B EC	cmp ebp, esp
...	...	...
300040104A	C3	ret

型数据占 32 位。请回答下列问题：

- (1) 计算 $f(10)$ 需要调用函数 `f1` 多少次？执行哪条指令会递归调用 `f1`？
- (2) 上述代码中，哪条指令是条件转移指令？哪几条指令一定会使程序跳转执行？
- (3) 根据第 16 行的 `call` 指令，第 17 行指令的虚拟地址应是多少？已知第 16 行的 `call` 指令采用相对寻址方式，该指令中的偏移量应是多少（给出计算过程）？已知第 16 行的 `call` 指令的后 4 字节为偏移量，M 是采用大端方式还是采用小端方式？
- (4) $f(13) = 6227020800$ ，但 `f1(13)` 的返回值为 1932053504，为什么两者不相等？要使 `f1(13)` 能返回正确的结果，应如何修改 `f1` 的源程序？
- (5) 第 19 行的 `imul` 指令（带符号整数乘）的功能是 $R[ecx] \leftarrow R[ecx] \times R[ecx]$ ，当乘法器输出的高、低 32 位乘积之间满足什么条件时，溢出标志 $OF = 1$ ？要使 CPU 在发生溢出时转异常处理，编译器应在 `imul` 指令后应加一条什么指令？

(1) 调用。求 $f(10)$ 的 `f1` 调用实例共有 10 次（10 到 1）；其中源码第 16 行的递归 `call f1` 执行 9 次。第 16 行 `call` 是递归调用指令。

(2) 跳转。第 12 行 `jle` 是条件转移（ $n \leq 1$ 走基例）；第 20 行 `jmp` 是无条件转移，执行到此处一定改变控制流；第 16 行 `call` 也必跳到被调用函数并保存返回位置。

(3) 下一地址与编码。第 16 行地址 00401025H，5 字节长，下一条 0040102AH（第 17 行）。目标 00401000H，位移为 $00401000 - 0040102A = -2AH = FFFFFFFD6H$ 。机器码 E8 D6 FF FF FF；偏移量在低地址先存 D6，所以是小端。

(4) 13! 不同。13!=6227020800 超32位有符号范围；低32位 7328CC00H，按题设回绕解释为 1932053504。要返回正确值，递归函数及乘积的计算/返回类型都改为足够宽的整数，如 long long f1(int n){ return n>1 ? (long long)n*f1(n-1) : 1LL; }。仅把最后的接收变量变宽，前面的32位乘法已经失真。

(5) OF 与异常。signed imul 的64位数学乘积若不能由低32位作符号扩展恢复，高32位便不等于低32位符号位的全0/全1扩展，OF=1。要在该条件转异常处理，可在 imul 后用 jo (溢出则跳)。

“调用次数”与“call 执行次数”分账

最外层调用 f1(10) 也是一次函数调用，但它不是函数体第16行执行出的；只要把 10,9,...,1 写下来，就能分别数出10个栈帧与9次自调用。位移一律从当前指令末尾算，不是从 opcode 地址算。

04 | 2021-43：格式字段各自约束什么

43. (15 分) 假定计算机 M 字长为 16 位，按字节编址，连接 CPU 和主存的系统总线中地址线为 20 位、数据线为 8 位，采用 16 位定长指令字，指令格式及其说明如下：

格式	6 位	2 位	2 位	2 位	4 位	指令功能或指令类型说明
R 型	000000	rs	rt	rd	opl	$R[rd] \leftarrow R[rs] \text{ opl } R[rt]$
I 型	op2	rs	rt	imm		含ALU运算、条件转移和访存操作3类指令
J 型	op3	target				PC 的低 10 位 $\leftarrow$ target

其中，op1~op3 为操作码，rs, rt 和 rd 为通用寄存器编号，R[r]表示寄存器 r 的内容，imm 为立即数，target 为转移目标的形式地址。请回答下列问题。

1) ALU 的宽度是多少位？可寻址主存空间大小为多少字节？指令寄存器、主存地址寄存器 (MAR) 和主存数据寄存器 (MDR) 分别应有多少位？

2) R 型格式最多可定义多少种操作？I 型和 J 型格式总共最多可定义多少种操作？通用寄

存器最多有多少个？

3) 假定 op1 为 0010 和 0011 时，分别表示带符号整数减法和带符号整数乘法指令，则指令 01B2H 的功能是什么 (参考上述指令功能说明的格式进行描述)？若 1, 2, 3 号通用寄存器当前内容分别为 B052H, 0008H, 0020H，则分别执行指令 01B2H 和 01B3H 后，3 号通用寄存器内容各是什么？各自结果是否溢出？

4) 若采用 I 型格式的访存指令中 imm (偏移量) 为带符号整数，则地址计算时应对 imm 进行零扩展还是符号扩展？

5) 无条件转移指令可以采用上述哪种指令格式？

(1) 宽度与容量。字长16位，ALU 操作数宽 16位；20位字节地址可寻址 2^{20} B=1MB；MAR需 20位 承载地址，MDR按8位数据总线为 8位。指令本身16位，取指可分两次经8位总线，不能因此把 MDR 误写成 16位。

(2) 操作与寄存器。R型 op1 4位,可定义 16种;I/J型6位主操作码,000000 留给 R,合计最多 63种;寄存器字段2位可编码 4个通用寄存器。

(3) 解码与运算。01B2H=000000 01 10 11 0010: R型 rs=1,rt=2,rd=3,op1=0010,故 $R3 \leftarrow R1 - R2$ 。R1=B052H、R2=0008H 时结果 B04AH,在 signed16 范围内;01B3H 的 op1=0011 是加法,结果 B05AH,也未溢出。初始 R3 的 0020H 被覆盖。判断 OF 看 signed 输入与结果的符号关系,而不是见最高位为1就报溢出。

(4) I型访存偏移。若 imm 为有符号偏移,应先符号扩展再与基址相加;零扩展会把负偏移错误解释为大正数。

(5) 无条件跳转。J型带 target,把形式地址写到 PC 低10位;它就是题中无条件转移的格式。I型也可编码条件转移,但不能在没有额外约束时直接把 I 型的所有指令说成无条件跳转。

位宽不要互相借

MAR 对应地址线20位;MDR对应数据线8位;ALU与通用寄存器对应字长16位。题面分别给这些值是为了让你判断它们属于不同路径,不是让一个“机器字长”统管全部字段。

05 | 2023-44: 按机器码长度走,才能正确取下一条

44. (9 分) 题 43 中 C 程序段在计算机 M 上的部分机器级代码如下,每个机器级代码行中依次包含指令序号、虚拟地址、机器指令和汇编指令。

for (i = 0; i < 24; i++)			
1	00401072	C7 45 F8 00 00 00 00	mov[ebp - 8], 0
2	00401079	EB 09	jmp 00401084h
3	0040107B	8B 55 F8	mov eax, [ebp - 8]
.....			
7	00401088	7D 32	jge 004010bch
for (j = 0; j < 64; j++)			
8	0040108A	C7 45 FC 00 00 00 00	mov[ebp - 4], 0
.....			
a[i][j] = 10;			
.....			
19	004010AE	C7 84 82 00 20 42 00 0A 00 00 00	mov[ecx + edx * 4 + 00422000h], 0Ah
20			

请回答下列问题。

- (1) 第 20 条指令的虚拟地址是多少?
- (2) 已知第 2 条 jmp 和第 7 条 jge 都是跳转指令,其操作码分别是 EBH 和 7DH,跳转目标地址分别为 0040 1084H 和 0040 10BCH,这两条指令分别采用什么寻址方式?请给出第 2 条指令 jmp 的跳转目标地址计算过程。
- (3) 已知第 19 条 mov 指令的功能是“ $a[i][j] \leftarrow 10$ ”,其中 ecx 和 edx 为寄存器名,0042 2000H 是数组 a 的首地址,指令中源操作数采用什么寻址方式?已知 edx 中存放的是变量 j,ecx 中存放的是什么?根据该指令的机器码判断计算机 M 采用的是大端还是小端方式?
- (4) 第 1 次执行第 19 条指令时,取指令过程中是否会发生缺页异常?为什么?

(1) 第20条地址。第19条起址 004010AEH，机器码 C7 84 82 00 20 42 00 0A 00 00 00 共 11 字节，所以下一条（第20条）为 004010B9H。逐个按图中的实际字节数计，不凭汇编语句长短猜。若肉眼计数，分组为 C7(1)+84(1)+82(1)+00 20 42 00(4)+0A 00 00 00(4)=11。

(2) 跳转。第2条 EB 09 是短相对位移，下一地址 0040107BH，加 09H 得 00401084H；第7条 7D 32 同为短相对位移，下一地址 0040108AH，加 32H 得 004010BCH。相对量的基点是各自下一条指令。

(3) 有效地址。mov [ecx+edx*4+00422000H],0AH 是带比例的基址/变址加位移内存寻址；ecx 保存行号 i 对应的字节偏移 $i \times 64 \times 4$ ，edx 保存列号 j，首址为 00422000H。位移字节 00 20 42 00 对应 00422000H、立即数 0A 00 00 00 对应 10，均按小端排。

(4) 首次执行第19条。其指令位于已执行过前面同一代码页 00401000H，所以取指本身不因首次执行该条而缺页；首次写 a[0][0] 可能引发数据页缺页，二者要分开。第19条全11字节从 ...0AE 到 ...0B8，未跨4KB代码页边界。

只记住一个动作

先把机器码划成 opcode、寻址字节、位移、立即数四段，再求“起址+总字节数”；位移跳转再以这个下一地址为起点。数组下标由行长64和元素4字节翻译为字节地址。

06 | 2024-44：小端字节还原元素，页号用地址高位

44. (10 分) 对于题 43 中的计算机 M，C 语言程序 P 包含的语句 “sum+ = a[i];”

在 M 中对应的指令序列 S 如下。

slli r4, r2, 2	//R[r4]←R[r2]<<2
add r4, r3, r4	//R[r4]←R[r3]+R[r4]
lw r5, 0(r4)	//R[r5]←M[R[r4]+0]
add r1, r1, r5	//R[r1]←R[r1]+R[r5]

已知变量 i、sum 和数组 a 都为 int 型，通用寄存器 r1~r5 的编号为 01H~05H。

请回答下列问题。

(1) 根据指令序列 S 中每条指令的功能，写出存放数组 a 的首地址、变量 i 和 sum 的通用寄存器编号。(3 分)

(2) 已知 M 为小端方式计算机，采用页式存储管理方式，页大小为 4KB。若执行到指令序列 S 中第 1 条指令时，i=5 且 r1 和 r3 的内容分别为 0000 1332H 和 0013 DFF0H，从地址 0013 DFF0H 开始的存储单元内容如题 44 图所示，则执行 “sum+=a[i];” 语句后，a[i]的地址、a[i]和 sum 的机器数分别是什么（用十六进制表示）？a[i]所在页的页号是多少？此次执行中，数组 a 至少存放在子页中？(5 分)

地址	0	1	2	3	4	5	6	7
0013 DFF0	FF	FF	FF	7C	70	FE	FF	FF
0013 DFF8	00	00	00	0C	3C	02	01	FF
0013 E000	F0	F1	00	00	DC	EC	FF	FF
0013 E008	FF	FF	01	02	00	00	01	02

题 44 图

(3) 指令 “slli r4, r2, 2” 的机器码是什么（用十六进制表示）？若数组 a 改为 short 类型，则指令序列 S 中 slli 指令的汇编形式应是什么？(2 分)

此题的 slli 编码依赖2024-43 给出的指令格式，不能只看本题截图凭常见 MIPS/RISC-V 格式猜。

(1) 寄存器。slli r4,r2,2 把 i 左移两位生成字节偏移，add r4,r3,r4 因此 R3 为 数组 a 首址、R2 为 i；lw 读 a[i]，add r1,r1,r5 因此 R1 为 sum。

(2) 按地址取数。首址 0013DFF0H，i=5，元素地址 +5×4=0013E004H；该处四字节 DC EC FF FF 按小端还原为 FFFFECDCH。初始 sum 00001332H，相加后低32位 0000000EH。4KB页号为地址右移12位，0013E004H 的页号 0013EH；此次至少需要 a[0] 到 a[5] 六个 int，从 ...DFF0 起的前四个占该页末16B，后两个落下一页，因此数组 a 至少跨2页。题未给完整数组长度，不能推出实际最多跨几页。

(3) 编码/改 short。原题43给的位段依次 000000 | shamt(2) | rs1(2) | 010 | rd(4) | 0010011，合成 00212213H。若 a 的元素变 short，占2B，计算 a[i] 字节偏移应左移1位，汇编变 slli r4,r2,1；其余操作是否改为 short 访存还需与题设完整程序配套，本小问只问这条移位形式。

为何至少两页

数组不是从页首开始，而从...DFF0 距页末16B；四个4B元素正好填满第一页，a[4]、a[5]已进入下一页。题只给本次 i=5，没有给数组总长度，切勿借用另一道题的数组规模。

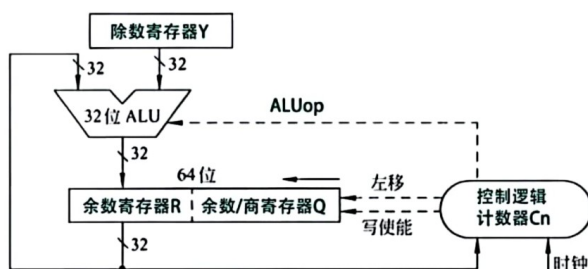
07 | 2025-44：补码除法的数据通路 with 异常

44. (本题 9 分)

对于题 43 中计算机 M 和程序 P，假定 P 的部分机器级代码如题 44 (a) 图所示，其中，R0~R4 为通用寄存器，SEXT 表示按符号扩展；M 中补码除法器逻辑结构如题 44 (b) 图所示。

```
.....
// X 在 R2 中, i 在 R4 中
//数组 d 的首地址在 R3 中
mov R1, (R3 + R4*4)      // R1 <- d[i]
scov R1                  //{R0, R1} <- SEXT(R1)
idiv R1,R3               //R1 <- ({R0, R1}/R2)
.....
```

题 44 (a) 图 程序代码



题 44 (b) 图 补码除法器逻辑结构

请回答下列问题，并说明理由。

(1) 若执行题 44 (a) 图中 idiv 指令的除运算时，d[i]=0x87654321, x=0xff，则补码除法器中寄存器 R、Q 和 Y 的初始内容分别是什么（用十六进制表示）？题 44 (b) 图中哪个部件包含计数器？在补码除法器执行过程中，由 ALUop 所控制的 ALU 运算有哪几种？（4 分）

(2) 假设 idiv 指令执行过程中会检测并触发除法异常，则执行 idiv 指令时，哪些情况下会发生除法异常（要求给出此时 d 用和 x 的十六进制表示机器数）？发生除法异常时，在异常响应过程中 CPU 需要完成哪些操作？（5 分）

原图需先核对的冲突。文字说 x 在 R2、数组首址在 R3；idiv R1,R3 这行的旁注却写 {R0,R1}/R2。按字面 R3 是数组地址，无法得到题目要求的 x 除数。本题以下依文字与旁注共同指向的 R2=x 解释，不把 R3 地址当作 x。

(1) 初始寄存器。d[i]=87654321H 最高位1，作为 signed32 是负数；scov 将其符号扩展到 {R0,R1}，故除法器初始余数寄存器 R=FFFFFFFFH、商/被除数低部 Q=87654321H，除数寄存器 Y=000000FFH。控制逻辑里的 Cn 是迭代计数器；每一位除法由 ALU 作加或减（配合移位与符号判定），而非普通一次算出商。

(2) 异常。两类型输入：x=00000000H 时除0，d 可取任意值如 00000001H；d=80000000H、x=FFFFFFFFH 时最小负数除-1，数学商 2^{31} 不可装入 signed32。CPU 检测后保存异常现场（断点 PC/状态及原因），切换到内核/异常态并跳到异常向量，由处理程序决定终止、报告或恢复。除法器不应把溢出的商悄悄写回当作正常结果。

从符号扩展理解 R 与 Q

32位负数扩成64位，左半部补32个1，右半部保留原位型；除数 0xFF 在题目给的 x=0xff 是正的255，不能误当8位 signed 的-1。R/Q/Y 的初始化是进入迭代之前的机器状态，不是最终商和余数。

混合限时复做

用“位型→解释→操作→保存→标志”写 01、02、03、07；用“指令长度→下一地址→有效地址→页内偏移”写 04、05、06。任何一次停住都记录具体断点，再展开该题旁的机制；目前尚无用户独立限时证据。